

P2: Teaching an Agent to Play — From Tabular Q-Learning to Neural Q-Networks

Prerequisites: P1 (the game engine). You will need Python 3.9+, NumPy, and PyTorch. All code in this post builds on the GameState, apply_move, get_legal_moves, encode_state, and get_legal_move_mask functions from P1.

T2 introduced the theory of TD learning, Q-learning, and policy gradients. This post implements them directly on Ultimate Tic-Tac-Toe and watches what happens. The story is a honest one: tabular methods fail on this game almost immediately, for a reason that turns out to be informative. Neural Q-learning does better, but still hits a ceiling. Understanding exactly where each approach breaks down is the motivation for the AlphaZero architecture in P3.

1. Tabular Q-Learning: The State-Space Problem

Q-learning maintains a table mapping each (state, action) pair to an expected future reward. On small problems — a 4×4 grid world, a simple maze — this works well. The table is finite and manageable.

For UT TT, the state space is bounded by $3^{81} \approx 4.4 \times 10^{38}$ distinct board configurations. A Q-table with one float per entry, if it could be stored at all, would require more storage than exists on Earth.

We can still implement tabular Q-learning — it just learns only from the specific positions it visits, generalising to nothing it hasn't seen before. Let's do this and measure how far it gets.

```
import random
import numpy as np
from collections import defaultdict

class TabularQAgent:
    def __init__(self, epsilon=0.2, alpha=0.1, gamma=0.99):
        self.epsilon = epsilon # exploration rate
        self.alpha = alpha # learning rate
        self.gamma = gamma # discount factor
        # Q-table: maps (state_key, move_idx) → float
```

```

# defaultdict initialises unseen entries to 0.0
self.q = defaultdict(float)

def state_key(self, state):
    """Convert GameState to a hashable key."""
    return (state.cells.tobytes(),
            state.sub_board_results.tobytes(),
            state.active_sub_board)

def select_action(self, state):
    legal = get_legal_moves(state)
    if random.random() < self.epsilon:
        return random.choice(legal)
    key = self.state_key(state)
    # Pick the legal action with the highest Q-value
    return max(legal, key=lambda m: self.q[(key, m)])

def update(self, state, move, reward, next_state, done):
    key = self.state_key(state)
    next_key = self.state_key(next_state)
    current_q = self.q[(key, move)]

    if done:
        target = reward
    else:
        legal_next = get_legal_moves(next_state)
        if not legal_next:
            target = reward
        else:
            best_next = max(self.q[(next_key, m)] for m in legal_next)
            target = reward + self.gamma * best_next

    # TD update
    self.q[(key, move)] += self.alpha * (target - current_q)

```

Training loop

To train, we run self-play games where one agent plays as X and a copy (or a random agent) plays as O. After each move, we compute a reward and update:

```
def play_training_game(agent_x, agent_o, reward_win=1.0, reward_loss=-1.0, reward_draw=0.0):
    """Play one game and update both agents. Returns winner."""
    state = GameState()
    # Track (state, move) pairs so we can do the final-outcome update
    trajectory_x = []
    trajectory_o = []

    while not state.is_terminal:
        if state.current_player == 1:  # X's turn
            move = agent_x.select_action(state)
            trajectory_x.append((state, move))
        else:  # O's turn
            move = agent_o.select_action(state)
            trajectory_o.append((state, move))
        state = apply_move(state, move)

    # Assign terminal rewards
    if state.winner == 1:
        rx, ro = reward_win, reward_loss
    elif state.winner == -1:
        rx, ro = reward_loss, reward_win
    else:
        rx = ro = reward_draw

    # Update X's trajectory (only terminal reward – no bootstrapping at end)
    for s, m in reversed(trajectory_x):
        agent_x.update(s, m, rx, state, done=True)
        rx *= agent_x.gamma  # discount backward through the trajectory

    for s, m in reversed(trajectory_o):
        agent_o.update(s, m, ro, state, done=True)
        ro *= agent_o.gamma
```

```
return state.winner
```

Evaluation

After training, we measure win rate against a random agent:

```
def evaluate_vs_random(agent, n_games=200, play_as=1):
    """Returns (win_rate, draw_rate, loss_rate)."""
    wins = draws = losses = 0
    for seed in range(n_games):
        random.seed(seed)
        state = GameState()
        while not state.is_terminal:
            if state.current_player == play_as:
                move = agent.select_action(state)
            else:
                move = random.choice(get_legal_moves(state))
            state = apply_move(state, move)
        if state.winner == play_as:
            wins += 1
        elif state.winner == 0:
            draws += 1
        else:
            losses += 1
    total = n_games
    return wins/total, draws/total, losses/total
```

Results

Training the tabular agent for 5,000 games:

```
agent_x = TabularQAgent(epsilon=0.3, alpha=0.1, gamma=0.99)
agent_o = TabularQAgent(epsilon=0.3, alpha=0.1, gamma=0.99)

for episode in range(5000):
    # Decay exploration over time
    eps = max(0.05, 0.3 * (1 - episode / 5000))
    agent_x.epsilon = agent_o.epsilon = eps
```

```

play_training_game(agent_x, agent_o)

print(f"Q-table size: {len(agent_x.q):,} entries")
wr, dr, lr = evaluate_vs_random(agent_x)
print(f"Win/Draw/Loss vs random: {wr:.1%} / {dr:.1%} / {lr:.1%}")

```

Typical output:

Q-table size: 142,847 entries

Win/Draw/Loss vs random: 54.0% / 7.5% / 38.5%

The agent visited 142,847 (state, move) pairs across 5,000 games. That is a tiny fraction of the full state space. Against a random opponent, it manages 54% wins — slightly better than the ~50% that random play achieves. It has not generalised at all to positions it hasn't seen.

More training makes this worse, not better: the Q-table grows linearly with games played, memory usage climbs, and the win rate plateaus because new games keep visiting new states that share no information with visited ones.

The fundamental problem: tabular methods cannot generalise. Seeing position A tells you nothing about position B, even if they differ by only one move. For UTTT, nearly every position encountered is novel.

2. Neural Q-Learning: Function Approximation

The fix is to replace the Q-table with a neural network that maps board states to Q-values. The network can generalise — positions that look similar (as tensors) will receive similar Q-value estimates, even if they have never been seen before.

We use the 7-channel `encode_state` tensor from P1 as input and output 81 Q-values, one per possible move.

The Q-Network

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class QNetwork(nn.Module):

```

```

def __init__(self):
    super().__init__()
    # 3 conv layers over the (7, 9, 9) input
    self.conv = nn.Sequential(
        nn.Conv2d(7, 32, kernel_size=3, padding=1), nn.ReLU(),
        nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
        nn.Conv2d(64, 64, kernel_size=3, padding=1), nn.ReLU(),
    )
    # Fully connected head: 64*9*9 = 5184 → 256 → 81
    self.fc = nn.Sequential(
        nn.Flatten(),
        nn.Linear(64 * 9 * 9, 256), nn.ReLU(),
        nn.Linear(256, 81),
    )

def forward(self, x):
    return self.fc(self.conv(x))

```

The convolutional layers are appropriate here because the board has spatial structure: the 3×3 arrangement of sub-boards, and the 3×3 cells within each sub-board, both matter. A convolutional filter can detect local patterns (a row in a sub-board, a diagonal) regardless of where on the 9×9 grid they appear.

The DQN Agent

Neural Q-learning (DQN) adds two key ingredients over plain Q-learning: an **experience replay buffer** that stores past transitions and samples from them randomly, and a **target network** that provides stable Q-value targets during training.

```

from collections import deque

class ReplayBuffer:
    def __init__(self, capacity=50_000):
        self.buffer = deque(maxlen=capacity)

    def push(self, state_tensor, move, reward, next_tensor, done, legal_mask):
        self.buffer.append((state_tensor, move, reward, next_tensor, done, legal_mask))

```

```

def sample(self, batch_size):
    batch = random.sample(self.buffer, batch_size)
    states, moves, rewards, nexts, dones, masks = zip(*batch)
    return (torch.stack(states),
            torch.tensor(moves, dtype=torch.long),
            torch.tensor(rewards, dtype=torch.float32),
            torch.stack(nexts),
            torch.tensor(dones, dtype=torch.float32),
            torch.stack(masks))

```

```

def __len__(self):
    return len(self.buffer)

```

```

class DQNAgent:

```

```

    def __init__(self, lr=1e-3, gamma=0.99, epsilon=0.5,
                 batch_size=256, target_update_freq=500):
        self.online_net = QNetwork()
        self.target_net = QNetwork()
        self.target_net.load_state_dict(self.online_net.state_dict())
        self.target_net.eval()

        self.optimizer = torch.optim.Adam(self.online_net.parameters(), lr=lr)
        self.buffer = ReplayBuffer()
        self.gamma = gamma
        self.epsilon = epsilon
        self.batch_size = batch_size
        self.target_update_freq = target_update_freq
        self.steps = 0

```

```

    @torch.no_grad()

```

```

    def select_action(self, state):
        legal = get_legal_moves(state)
        if random.random() < self.epsilon:
            return random.choice(legal)
        # Convert state to tensor

```

```

s = torch.tensor(encode_state(state)).unsqueeze(0)  # (1, 7, 9, 9)
q_vals = self.online_net(s).squeeze(0)  # (81,)
# Mask illegal moves to -inf before argmax
mask = torch.tensor(get_legal_move_mask(state))
q_vals = q_vals.masked_fill(mask == 0, float('-inf'))
return int(q_vals.argmax())

def store(self, state, move, reward, next_state, done):
    s = torch.tensor(encode_state(state))
    ns = torch.tensor(encode_state(next_state))
    mask = torch.tensor(get_legal_move_mask(next_state))
    self.buffer.push(s, move, reward, ns, done, mask)

def train_step(self):
    if len(self.buffer) < self.batch_size:
        return None

    states, moves, rewards, nexts, dones, masks = self.buffer.sample(self.batch_size)

    # Current Q-values for the actions taken
    q_current = self.online_net(states).gather(1, moves.unsqueeze(1)).squeeze(1)

    # Target Q-values (using target network)
    with torch.no_grad():
        q_next = self.target_net(nexts)
        q_next = q_next.masked_fill(masks == 0, float('-inf'))
        q_next_max = q_next.max(dim=1).values
        q_next_max[q_next_max == float('-inf')] = 0.0 # terminal states
        targets = rewards + self.gamma * q_next_max * (1 - dones)

    loss = F.mse_loss(q_current, targets)
    self.optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(self.online_net.parameters(), 1.0)
    self.optimizer.step()

```

```

self.steps += 1
if self.steps % self.target_update_freq == 0:
    self.target_net.load_state_dict(self.online_net.state_dict())

return loss.item()

```

Training Loop

```

def train_dqn(agent, n_episodes=20_000, eval_every=1000):
    results = []

    for episode in range(n_episodes):
        # Decay epsilon from 0.5 to 0.05
        agent.epsilon = max(0.05, 0.5 - 0.45 * (episode / n_episodes))

        state = GameState()
        while not state.is_terminal:
            move = agent.select_action(state)
            prev_state = state
            state = apply_move(state, move)

            # Intermediate reward: 0.0 (only terminal states carry signal)
            # Give a small shaping reward for winning a sub-board
            intermediate = 0.0
            sb_won = state.sub_board_results[decode_move(move)[0]]
            if sb_won == prev_state.current_player:
                intermediate = 0.05

            done = state.is_terminal
            if done:
                reward = 1.0 if state.winner == prev_state.current_player else \
                    -1.0 if state.winner == -prev_state.current_player else 0.0
            else:
                reward = intermediate

            # The opponent also plays; from the agent's perspective that is

```

```

# an environment transition, not a separate agent step.
# We store the agent's own transitions only.
agent.store(prev_state, move, reward, state, done)
agent.train_step()

# Opponent plays a random move (we train as both X and O)
if not state.is_terminal:
    opp_move = random.choice(get_legal_moves(state))
    prev_opp = state
    state = apply_move(state, opp_move)
    # Store opponent transition from opponent's perspective
    done = state.is_terminal
    opp_reward = 1.0 if state.winner == prev_opp.current_player else \
        -1.0 if state.winner != 0 else 0.0
    agent.store(prev_opp, opp_move, opp_reward, state, done)
    agent.train_step()

if (episode + 1) % eval_every == 0:
    agent.epsilon = 0.0 # greedy eval
    wr, dr, lr = evaluate_vs_random(agent, n_games=200)
    agent.epsilon = max(0.05, 0.5 - 0.45 * (episode / n_episodes))
    results.append((episode + 1, wr, dr, lr))
    print(f"Ep {episode+1:6d} | W {wr:.1%} D {dr:.1%} L {lr:.1%}")

return results

```

Results and the Ceiling

Training for 20,000 episodes (roughly 15 minutes on a CPU):

```

Ep 1000 | W 56.5% D 6.0% L 37.5%
Ep 5000 | W 64.0% D 7.0% L 29.0%
Ep 10000 | W 70.5% D 8.5% L 21.0%
Ep 20000 | W 73.5% D 8.5% L 18.0%

```

Progress is real but slow, and the curve flattens. Several problems compound:

Sparse rewards. The only strong signal is the terminal win/loss. Every move in a 40-move game receives reward 0, and only the last receives ± 1 . The network must

credit 40 decisions from a single scalar — exactly the credit-assignment problem T2 described. (The small sub-board shaping reward helps slightly but does not solve it.)

No lookahead. The Q-network evaluates each position in isolation. It cannot reason about sequences of moves. A position that looks neutral might be a forced win in three moves — but the network cannot see that without searching.

Single-opponent training. Training against a random opponent caps the level of play the agent needs to beat. The agent learns to exploit random mistakes but never learns to handle a competent opponent.

Bootstrapping instability. TD updates bootstrap from the target network, which is only a slightly older copy of the online network. In large action spaces with sparse rewards, this creates noisy, unstable Q-value estimates.

3. What Would Fix This

The three problems above have three corresponding answers, all from the AlphaZero playbook:

Sparse rewards → MCTS search targets. Instead of training on terminal game outcomes, AlphaZero trains the network on the output of a Monte Carlo tree search. The MCTS provides a local, dense signal: “at this specific position, these specific moves are worth exploring.” The network learns from search, not from game outcomes.

No lookahead → explicit tree search. MCTS builds a search tree at inference time. The network provides value and policy estimates at each node; the tree search does the lookahead. The separation of “what does this position look like?” (network) and “where should I search?” (MCTS) is the key architectural insight.

Single-opponent training → self-play. AlphaZero trains exclusively against itself. As the network improves, the opponent improves at the same rate. The agent always faces a competent opponent, and the training distribution continuously reflects the current level of play.

The neural Q-network we built is not wasted work. The QNetwork architecture — convolutional layers over the 7-channel tensor, fully connected head — is essentially the same as AlphaZero’s tower, minus the dual-head output (value + policy) and the residual connections. P3 will take this architecture, add residual blocks and the dual head, wire it to MCTS, and train it via self-play.

Theory: [T2 — The RL Landscape](#) explains TD learning, Q-learning, and the credit-assignment problem that motivates what comes next. [T3 — From Bandits to Trees](#) covers the UCB and PUCT theory that P3 implements.

Full notebook: A companion notebook for P2 (coming soon) will include the full training run, loss curves, visualisations of learned Q-values, and a head-to-head tournament between the tabular agent, the DQN agent, and random play.